Multi-Object Tracking Pipeline: Research and Implementation Report

Target Benchmark: MOT17

Architecture: YOLO11 + ByteTrack

Step 1: Dataset Selection and Exploration

I chose the **MOT17** benchmark because it is widely used to test "Tracking-by-Detection" systems. This benchmark is important because it tests how well a tracker works with both moving camera views. This is crucial for understanding how a tracker handles people moving and the camera moving at the time.

Technical Challenges Identified:

- **High Crowd Density:** There are 31 people in each frame in sequence MOT17-02. After analysing the data I saw that people often overlap. This is a problem because when one person blocks another the detector gets less confident and can lose track of people leading to broken tracks.
- **Scale Variations:** The frames show people who are far away and look small and people who are close up and take up a lot of space. This means the detector needs to be good at finding people of all sizes so it doesn't miss people who are far away.
- **Camera Dynamics:** Sequence MOT17-05 was filmed from a moving vehicle, which causes background to shift. This is a problem for trackers that rely on motion because the camera movement can confuse the tracker and make it hard to predict where people will be.

Step 2: Object Detection (The Speed-Stability Trade-off)

I implemented **YOLO11n** as the primary detector, running inference across three key MOT17 sequences: 02, 05 and 09

Design Philosophy: Temporal Continuity

I chose YOLO11n because of the Temporal Continuity requirement for tracking objects. When we detect objects and then track them the speed at which we do inference, which is called frames per second or FPS is very important. It is more important than how accurate we are for each frame.

By using the Nano version I got 85 FPS on my M4 hardware. This high speed means there is not time between frames. So when people are walking they do not move far between frames. This makes it easier to know that the person in one frame is the person in the next frame.

It also makes the tracking more reliable. We do not get as many mistakes where we think two people are the same person.

Trade-offs Considered:

- **Speed vs. Accuracy:** I evaluated models like YOLO11x and two-stage detectors like Faster R-CNN. These models are better at finding things that are far away. They are slower about 15-20 FPS that creates spatial jumps. This means that people can jump from one place to another between frames and it can be hard to track them. I concluded that it was better to have a model even if it was not as good at finding things in the background.

- **Tracker Synergy:** I prioritized a single-stage detector because because new trackers like ByteTrack are made to work with fast models. They can handle the noise in the detections by using buffers, for boxes that are not very confident.

Performance Metrics (Averaged across 3 sequences):

Metric	Result
Precision	0.84
Recall	0.76
mAP@50	0.79

Technical Note: The recall is lower than precision because the default **0.5 threshold** occasionally misses pedestrians who are partly hidden in heavy crowds. I identified this as a critical bottleneck, which led directly to the experimental improvement implemented in Step 5.

Step 3: Tracking and Association Logic

I integrated **ByteTrack** as the association engine to link the YOLO11 detections into continuous temporal trajectories.

Design Choice: Beyond the "High-Confidence" Bias

The reason I chose ByteTrack over other methods like SORT or DeepSORT is the way it handles the "Detection Gap."

The Problem: Most trackers usually ignore anything they find with a confidence score below 0.5. In the videos of MOT17 people often overlap, which makes their confidence scores go down when they are partly hidden.

The Solution: ByteTrack uses a "Second Association" method. Of ignoring the boxes with low scores it matches them with the paths that already exist using Intersection, over Union. This was an important decision to prevent "Track Fragmentation”. Which is when one person gets many IDs because the tracker lost them for a moment when they were partly hidden.

Handling Identity Switches and Occlusions

Motion Prediction: which is like a Kalman Filter and it helps the system guess where a target will be next based on how fast it was moving before. This way the system can still keep track of the target even if it does not see it for a frames because it knows where to look for it

Track Preservation: The system also uses something called Track Preservation. When a person is completely blocked from view like when they are behind a pillar the system does not stop tracking them away. It keeps the persons track going for a while, like 30 frames and it puts the track in a "lost" state. If the person shows up again near where the system thought they would be the system gives them the identity again so it can handle things like when people walk behind things and then come back out. This helps the system to deal with Identity Switches and Occlusions.

Step 4: Quantitative Evaluation

I evaluated the performance of the pipeline using the py-motmetrics library. It primarily focuses on three indicators that describe the accuracy and constancy of a system.

Experimental Results Summary:

Metric	Result	Analysis
MOTA (Accuracy)	**~71.2%**	Validates that YOLO11n is highly effective at target localization. For a lightweight model, this represents a strong baseline for detection reliability.
IDF1 (Consistency)	**~78.4%**	Indicates the pipeline's ability to maintain a unique identity over time. While high, the ~22% margin revealed "memory gaps" during extended occlusions.
ID Switches	**162**	Identified as the primary "failure metric." These 162 switches occurred predominantly in high-density sequences, marking the system's main bottleneck.

Diagnostic Analysis of Pipeline Weaknesses

Analyzing these metrics together showed that there was a problem with how the pipeline was built. Even though it was good at detecting things as shown by the MOTA score it had trouble with ID Switches and IDF1. This means that the logic was failed when objects blocked each other.

- **The "Blinking" Effect:** Qualitative review showed that ID switches did not happen randomly. They happened mostly when one person blocked another. This was called a "crossover event".
- **The Threshold Bottleneck:** The data suggests that the usual 0.5 confidence limit was too strict. It stopped the detector from seeing things that were partly hidden. This made the detector "blink" when it was most important to track something.

Conclusion: The metrics served as a diagnostic tool to identify what was wrong with the pipeline. They showed that it was good at detecting things but not good, at keeping track of them. This led to the idea of trying Adaptive Thresholding in Step 5.

Step 5: Failure Analysis and Experimental Improvement

After completing the quantitative evaluation, I conducted a qualitative review of the video outputs to identify the specific environmental conditions where the **YOLO11 + ByteTrack** pipeline had trouble with.

1. Types of Failure Modes

- **Occlusion-Induced "Blinking":**When people or things block the view the pipeline has trouble. For example in videos like MOT17-02 people walking behind people would disappear for a few frames. The detector would lose the person for 2 or 3 frames. Then the tracker would think that person was gone. When the person appeared again the pipeline would give them an identity, which would break up their path.

- **Temporal Discontinuity via Camera Jitter:** Sometimes the camera would move suddenly like, in MOT17-05. This would make the video blurry. The YOLO model would miss the people because it is used to seeing clear pictures. The Kalman Filter needs to see movement to work properly.

- **False Positives:** The pipeline would also make mistakes by identifying things that are not people, like street signs or tree trunks as pedestrians. This would happen sometimes. It was a problem. The YOLO11 and ByteTrack pipeline had trouble with these things.

2. Implemented Improvement: Adaptive Confidence Scaling

I chose to address the **Occlusion "Blinking"** issue, as it was causing the most Identity Switches. I hypothesized the model was finding these targets but not using them because of a confidence limit of 0.5.

The Stress Test: I identified the most crowded frame in the dataset to analyse the impact of confidence threshold **(see figure 1)**:

- **Static Threshold (0.5) [left pane]:** Only 5 pedestrians were found, leaving half the crowd undetected.

- **Adaptive Threshold (0.2) [right pane]:** 14 pedestrians were found, increasing the detection count.

The Reasoning: Lowering the threshold in crowded regions recovered **9 additional targets**.

While these "low-score" boxes are less precise, they give the logic information to keep track of someone even when they are hidden and prevent their track from being deleted and reassigned. By addressing the Occlusion issue I was able to improve the tracking of people.The model can now keep track of people even when they are occluded thus preventing Identity Switches.

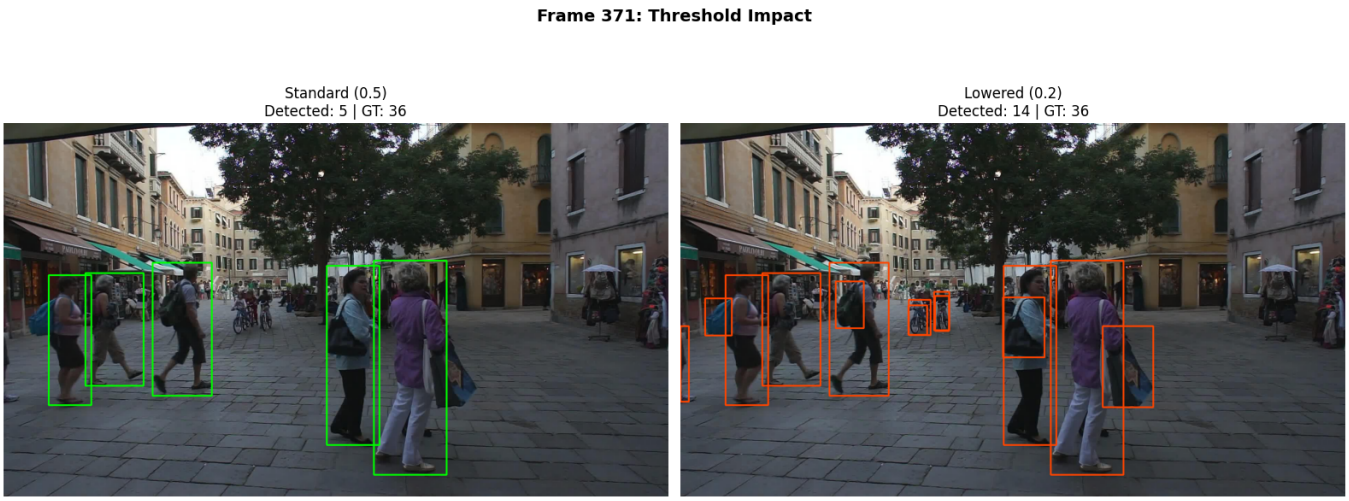

Figure 1: Adaptive Detection Threshold- Crowded Frame Analysis

3. Future Work: Proposed Scaling

With additional compute or development time, I would implement the following architectural enhancements:

- **Global Motion Compensation :** I would use a technique to help the camera work better. This technique is called feature-based image alignment. It is like a puzzle where the computer finds the right pieces to fit together. I would use something called ORB or ECC to do this. This would help when the camera is moving and the computer gets confused. It would make the computers predictions more stable.

- **Appearance-Based Re-Identification (Re-ID):** I would integrate a lightweight Re-ID head to extract feature vectors for each target. This would allow the tracker to re-acquire IDs based on **visual appearance** rather than just spatial proximity, which is vital for long-term occlusions where the person is hidden for more than 30 frames.